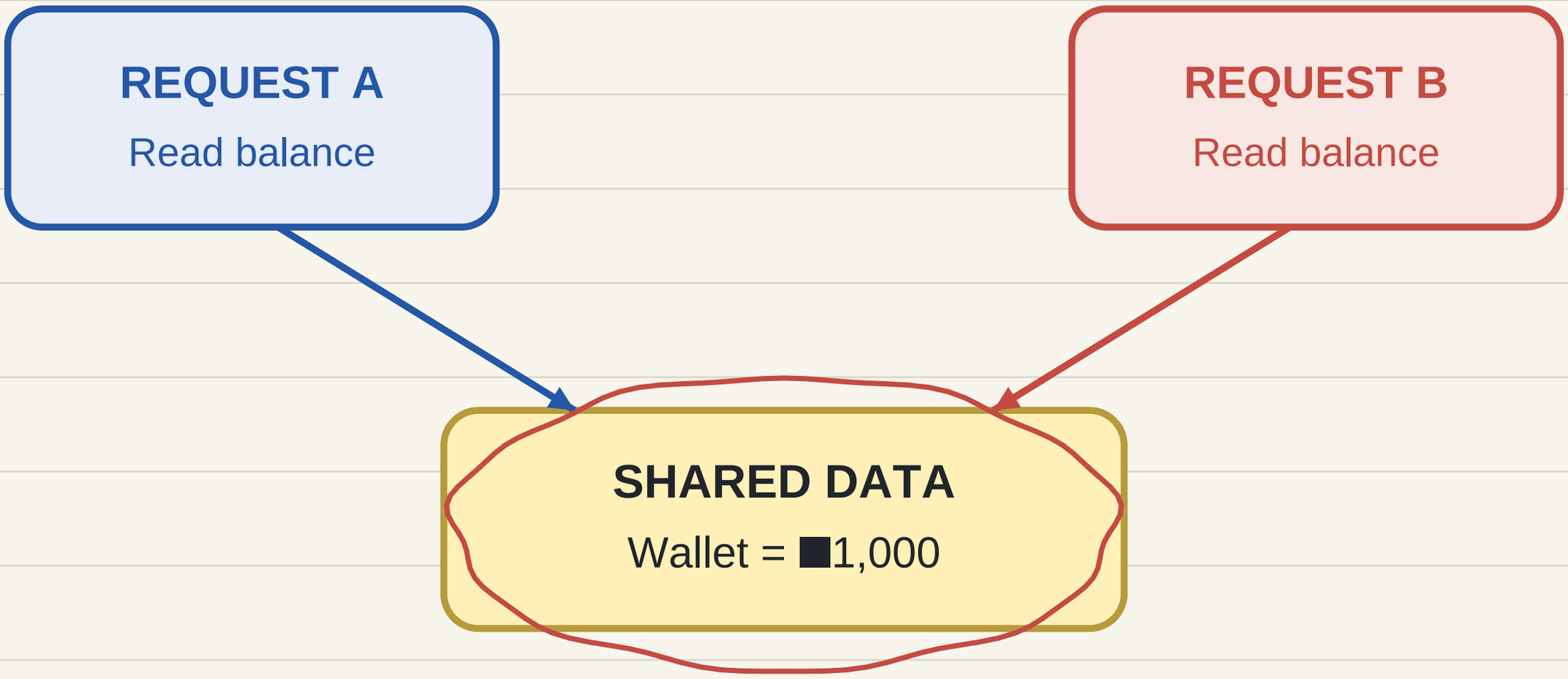
01 / 05 • BACKEND ENGINEERING

RACE CONDITIONS

Two requests. One shared piece of data. One very sneaky bug.



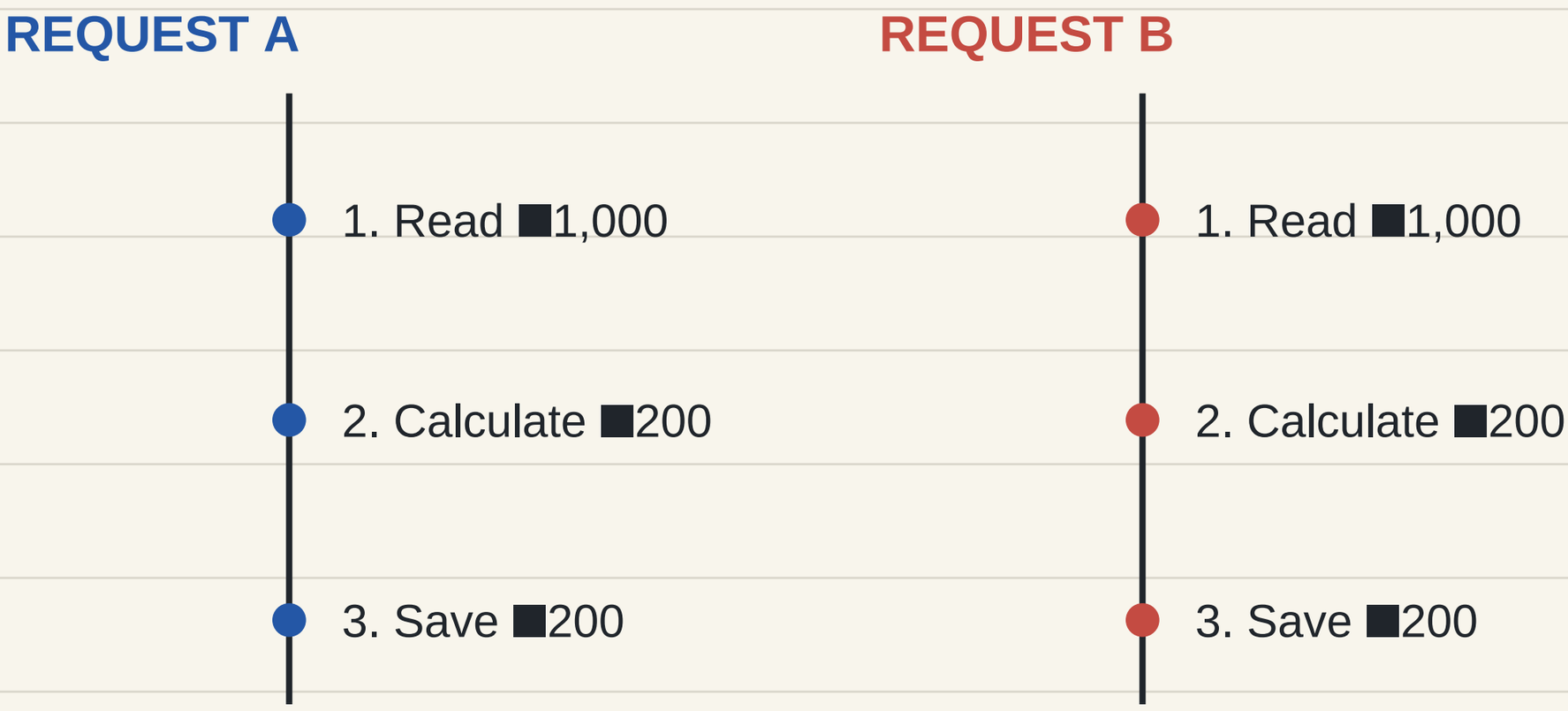
Swipe → to see how a race condition happens, and how to prevent it.

02 / 05 • THE IDEA

What is a race condition?

It happens when multiple requests access or change the same data at nearly the same time — and the result depends on which operation runs first.

SAME TIME ≠ SAFE

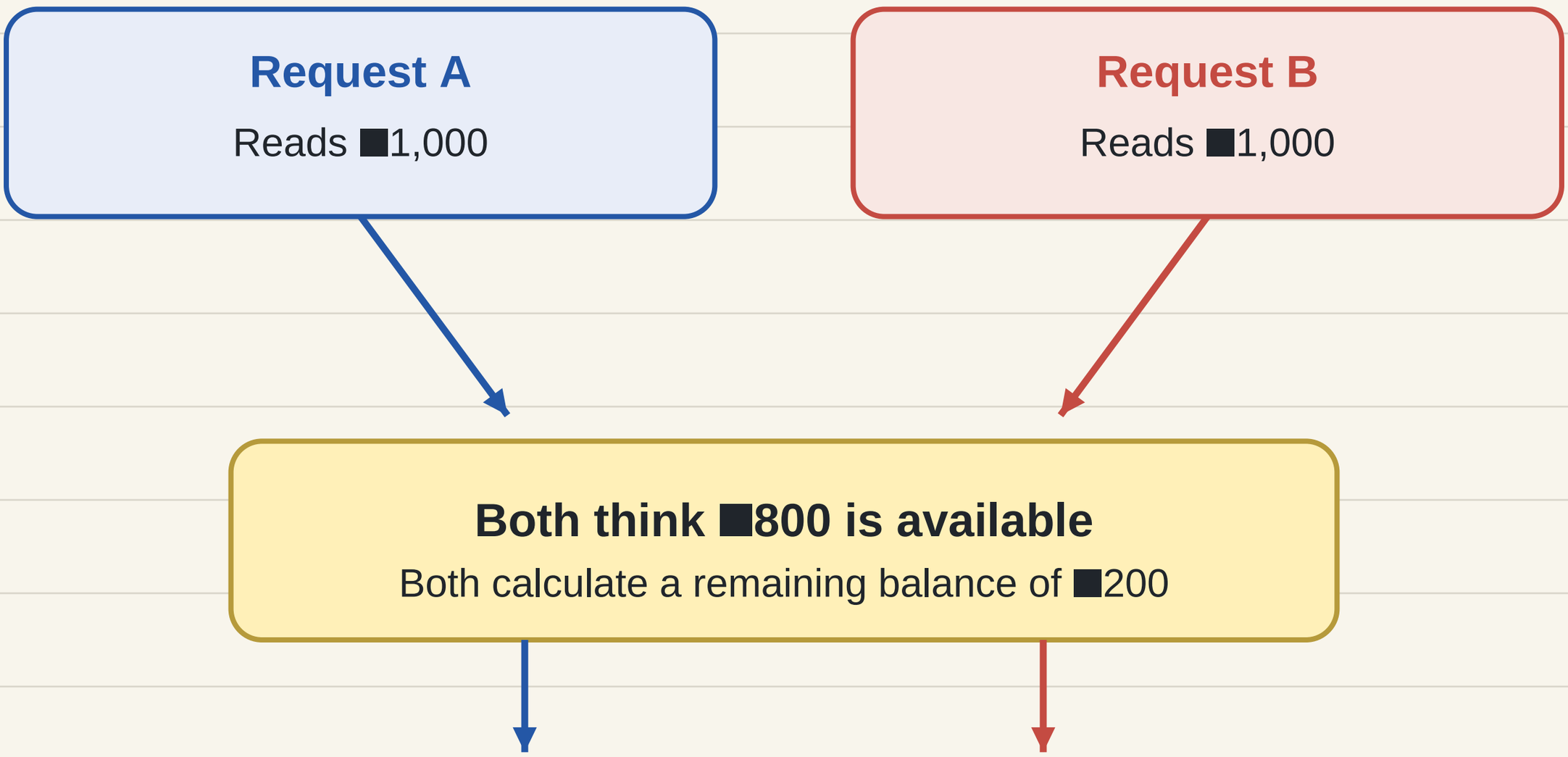


Both read old data → one update gets overwritten

03 / 05 • REAL-WORLD BUG

The wallet bug

Starting balance: ■1,000. Two withdrawals of ■800 arrive together.



The exact outcome depends on how the code and database handle concurrent updates.

04 / 05 • THE FIX

Make it safe

Don't rely on timing. Let the database protect the balance update.

```
UPDATE wallets
SET balance = balance - 800
WHERE user_id = 42
  AND balance >= 800;
```

First request

Updates balance to ■200

Second request

0 rows updated → reject

For real money movement, also use a transaction, handle the affected-row count, and keep the operation idempotent.

05 / 05 • REMEMBER THIS

The takeaway

01 **Shared data + concurrent requests**

A race condition can appear.

02 **Timing is not a safety strategy**

The bug may be rare — but costly.

03 **Use atomic updates / transactions**

Make the database enforce the rule.

Day 1/50 — Learning backend engineering

Next: Deadlocks • Follow along & build with me

I'm learning in public as a vibecoder — building products that are not just fast to ship, but safe and reliable.